Chapter 2: MySQL Database Integration

You will use MySQL as the database for this project. This chapter shows you how to set up your local development server, how to access it in Flask, and basic operations.

Installing MySQL

You will only need to go through this installation once. After that, you will create a new database for each project.

Windows Installation

1. Go to dev.mysql.com/downloads.
2. Download 'MySQL Installer for Windows' (`mysql-installer-community`).
3. Run the installer and select 'Custom' installation.
4. Select 'MySQL Server' ***and*** 'MySQL Workbench' from the products list.
5. Follow the wizard to complete installation.
6. Set a root password and remember it!

> ⚠ **If you did not enable the option to automatically start the server with every device boot, you will need to manually start it when you need it**
>
> - Open the Windows "Run" dialog by pressing the Windows key + R
> - Type `services.msc` and press "ok"
> - Find `MySQL80` in that list, right-click on it, and click "start"

Mac Installation

1. Go to dev.mysql.com/downloads.
2. Select macOS and download the DMG archives for `MySQL Community Server` ***and*** `MySQL Workbench` .
3. Double-click the downloaded DMG file.
4. Run the installer package (`.pkg`).
5. Follow the installation wizard.
6. **IMPORTANT:** Save the temporary root password shown at the end!

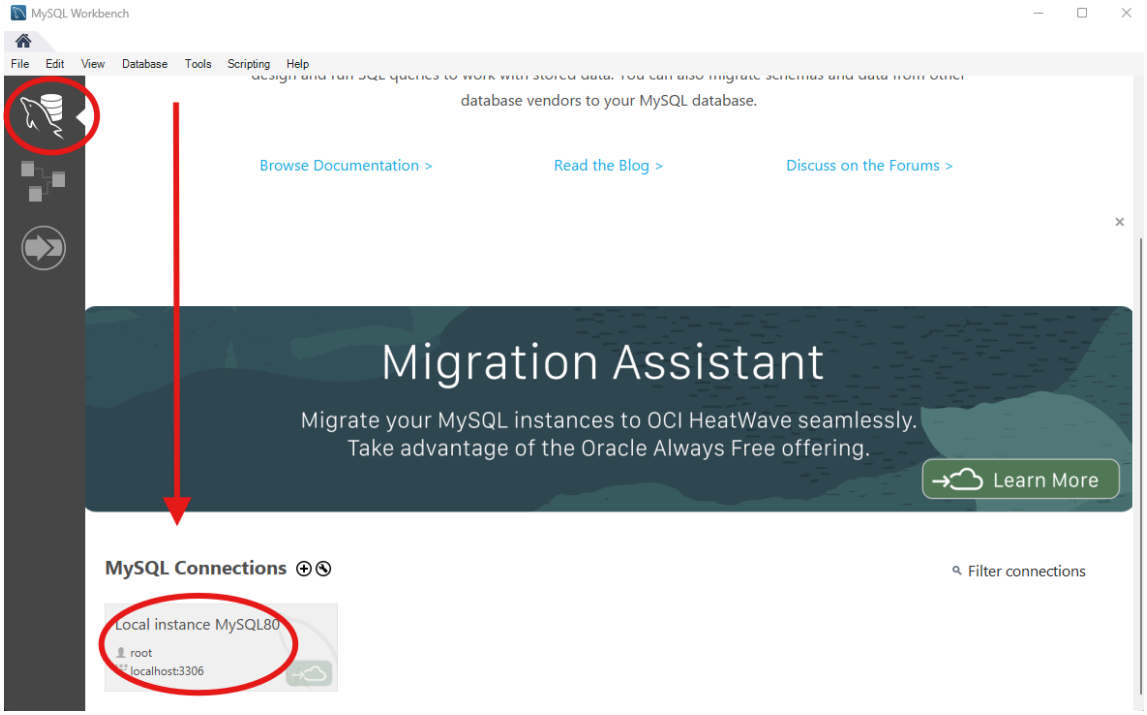
> ⚠ **If you lose your MySQL root password, you have to reset it through a recovery process.**

What are MySQL Server and Workbench?

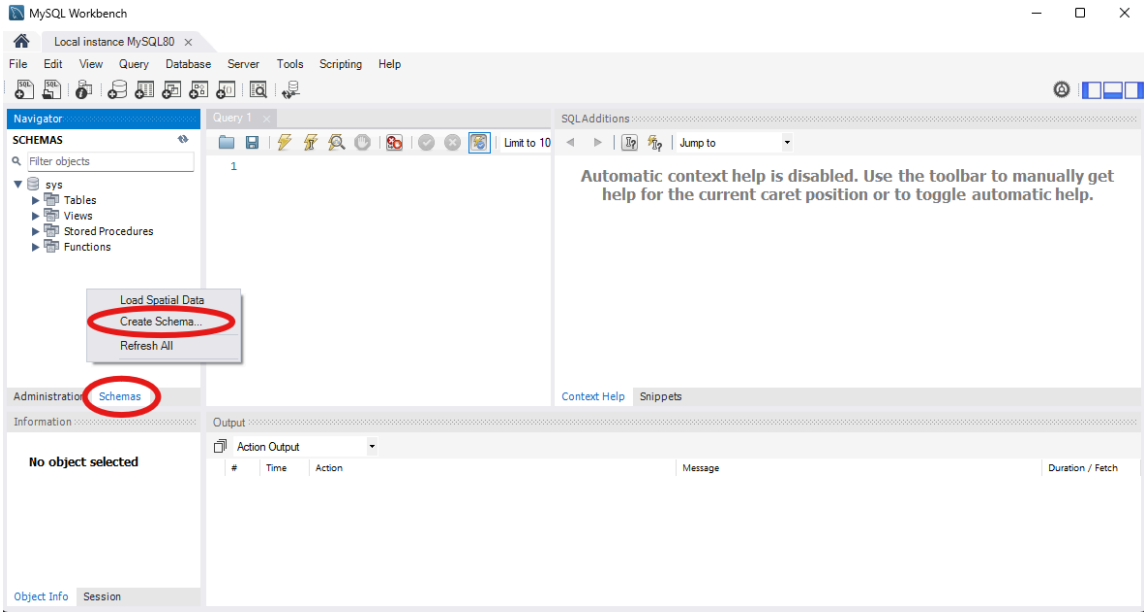
- The Server is what holds your databases, tables, entries.
- The Workbench is a program that you will use to look at the tables and verify that your flask app successfully stored data there.

Creating a database in MySQL Workbench

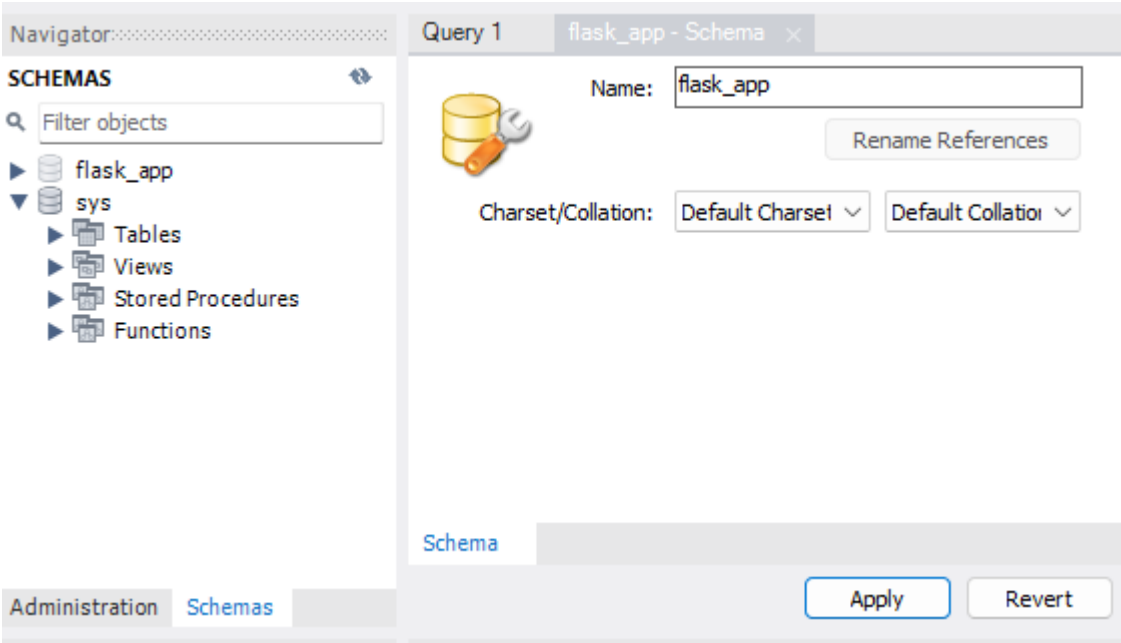
You will create a new database for each project. To create a new database, open the MySQL Workbench program and connect to your server (local instance) with the password you set during the MySQL installation.



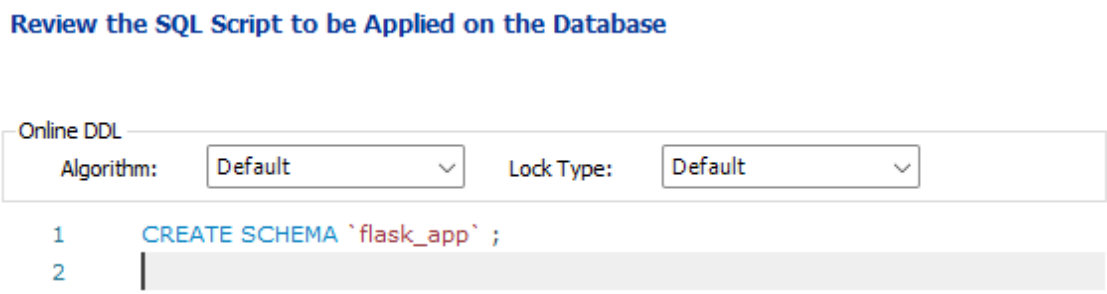
Then, on the left, switch to "Schemas", right click, and create a new schema.



Name the database. Because you will create a new database for each project, give it a more unique name than "flask_app".



MySQL Workbench will create the simple SQL script that creates the new database (schema). Execute that script and see your new database show up in the left panel.



You will not add or manipulate tables or entries through Workbench. The next sections will show you how to do that in Flask.

Configuring Flask-SQLAlchemy

Flask-SQLAlchemy is an object-relational mapper (ORM) that lets you interact with databases using Python objects instead of raw SQL queries. Install the required packages:

```
pip install flask flask-sqlalchemy pymysql
```

You will later connect your Flask app to the database with a code snippet similar to this:

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SECRET_KEY'] = 'your-secret-key-here'
```

In the above code, `app` is the Flask app instance. `.config['X'] = Y` sets a config variable X to a value Y. The imported SQLAlchemy library accesses this config to retrieve the database URI.

You will need to update the exact value for your own project. The *database connection string* is constructed as follows:

```
mysql+pymysql://username:password@hostname:port/database_name
```

- `mysql` is the database "dialect"
- `pymysql` is the database driver
- `root` is the username
- `password` is your root password
- `hostname` and `port` are your URL (localhost during development)
- `database_name` will be whatever you set as the name when you created the database

The secret key is used to cryptographically sign all communications. You can set it to any value.

Creating database models

You do not create tables in the MySQL Workbench. Instead, you will create tables and columns through the Flask app. To create a table, you create a data model as a Python class. Save the following code as your `app.py` and make sure you understand every single line of it:

app.py

```python
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model): ❶
    id = db.Column(db.Integer, primary_key=True) ❷
    username = db.Column(db.String(80), unique=True, nullable=False) ❸
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all() ❹
```

❶ The user model inherits from the `db.Model` superclass, telling the database to create a table for it.

2. This creates a column with the datatype integer and sets it as the primary key.
3. This creates a column with the datatype string that has to be unique, cannot be null (i.e., left out), and has a max length of 80 characters.
4. This creates all the tables when the app starts.

> **Choose your column types carefully!**
>
> Use integers for IDs because they can easily be incremented and searched. Set them as primary key so that you can reference users in other tables. Think about which other columns have to be unique and non-null. Consider limiting the length of strings. Read more about the available configurations below

SQLAlchemy column options

SQLAlchemy supports these common datatypes for columns:

Column Type	Description	Example
db.Integer	Whole numbers	id = db.Column(db.Integer)
db.String(n)	Text with max length n	name = db.Column(db.String(80))
db.Text	Long text (no length limit)	bio = db.Column(db.Text)
db.Float	Decimal numbers	price = db.Column(db.Float)
db.Boolean	True/False values	active = db.Column(db.Boolean)
db.DateTime	Date and time	created = db.Column(db.DateTime)

You might need these common parameters for your project:

- `primary_key=True` : Makes this column the primary key
- `unique=True` : Values must be unique across all rows
- `nullable=False` : Column cannot be empty (required field)
- `default=value` : Sets a default value if none provided

Flask Shell Operations

You can interact with your database directly from the Flask shell. This is useful for testing and debugging. In this command-line interface you write python code line by line into your terminal.

1. Start the Flask shell with `flask shell` instead of `flask run` to start your app. This allows you to continue working in the terminal.
2. Create a new user

```
new_user = User(username='john', email='john@example.com') ❶
db.session.add(new_user) ❷
db.session.commit() ❸
```

❶ This creates a new User object.

❷ This adds this user to the database does not yet commit (save) the change.

❸ This saves the change to the database.

3. Query all users

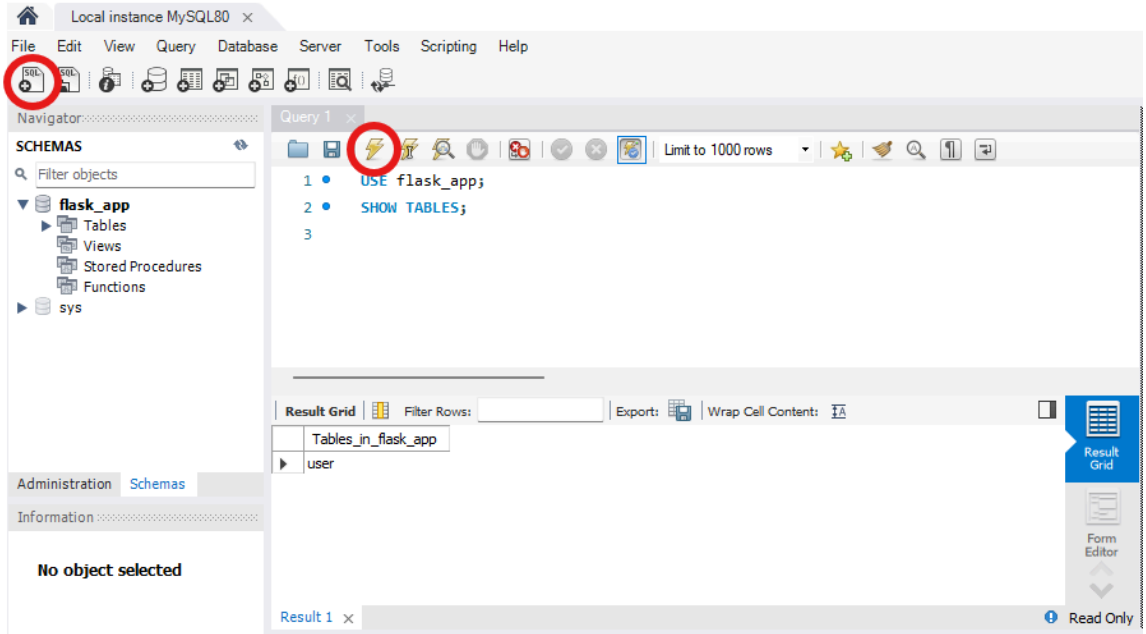
```
users = User.query.all()
for user in users:
    print(user.username, user.email)
```

Instead of `.all()` users you could query with filters (e.g., `.filter_by(username='john').first()`)

4. Exit the shell with `exit()`

Verifying changes in MySQL Workbench

You can inspect the database in MySQL Workbench. To create queries, click on "New Query Tab" and type in your query. To execute it, click the flash icon. You will see the results at the bottom. Start by selecting the correct database with `USE flask_app;`



- To view all tables in your database write `SHOW TABLES;`
- To see the structure of a table called `user` write `DESCRIBE user;`
- To see all entries in a table called `user` write `SELECT * FROM user;`
- To count how many entries are in the table `user` write `SELECT COUNT(*) FROM user;`

You can always come back to this program whenever you are unsure about the state of a specific table or the success of an operation.

Creating a simple read-only app

To create a quick visualization, put together all the snippets above in the following `app.py`. Only the highlighted lines are new.

app.py

```python
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all()

@app.route('/')
def index():
    users = User.query.all() 1
    return render_template('index.html', users=users) 2
```

1. This retrieves all the entries from the `User` table
2. This renders the template and passes the retrieved User objects to the template.

This server only has one route (`/`) and needs a new template. The function is already passing the `users` variable to the template. Create the `home.html` template as follows and see how it uses the passed variable.

templates/home.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard</title>
</head>
<body>
    <h1>List of all users</h1>
    <ul>
        <!-- List all entries retrieved from the database and passed to the template -->
        {% for user in users %}
            <li>{{ user.username }} - {{ user.email }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

With this set up, you have another way of seeing the changes you make to the table, either through the Flask shell or the methods that will be introduced below. Note that the list only refreshes on reload.

Flash messages

Flash messages are Flask's way of sending notifications to the browser to be displayed to the user (e.g., for errors or confirmations). Insert the following HTML snippet wherever you want the notifications to pop up (e.g., above or below the list of users). Read it thoroughly to make sure you understand every line.

```html
<!-- Flash messages -->
{% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
        {% for category, message in messages %}
            <div class="alert alert-{{ category }}">{{ message }}</div>
        {% endfor %}
    {% endif %}
{% endwith %}
```

- `get_flashed_messages` is a function provided by Flask to retrieved queued notifications.
- Set `with_categories` to true to include the type (e.g., error, success) in the messages.
- Iterate through the list of queued messages and create one `<div>` per message.
- These messages can be styled with the assigned css classes like `alert` and `alert-error`.

You can send messages to the client by adding `flash('Sorry, an error occurred', 'error')` on your server where the first string is the message and the second string the type. You will find examples of these in the following sections. Make sure to import the `flash` function from the Flask package at the top of your script!

Basic database operations (CRUD)

The four fundamental database operations as often called by their acronym "CRUD": create, read, update, delete. Below you will find implementations for each of these operations as different routes in the Flask application.

General explanations:

- Routes containing strings like `<int:user_id>` or `<string:email>` allow you to add parameters to your route. The format is `<datatype:variable_name>`.
- `db.session` is the object that manages all the changes you make to the database. The changes are only committed when you run `db.session.commit()` and can be reverted with `db.session.rollback()` if an error occurred.
- End every route with `return redirect(url_for('index'))` to bring visitors back to the home page, even though they clicked on a link (e.g., to `/add_user/`).

> **URL parameters are not the best method of passing variables and you will learn the correct way in the next chapter.**

CREATE - Add a new user

```
@app.route('/add_user/<string:username>/<string:email>') ①
def add_user(username, email):
    new_user = User(username=username, email=email) ②
    try:
        db.session.add(new_user) ③
        db.session.commit() ④
    except Exception as e:
        db.session.rollback() ⑤
    return redirect(url_for('index')) ⑥
```

① This endpoint requires a username and an email to be passed.

② This creates a new User object.

③ This adds the user to the database without committing the change yet.

④ This commits the change, only if there were no exceptions in the line before (e.g., username already exists).

⑤ If an exception occurred, this code rolls back the change (i.e., undo adding the user).

⑥ Regardless of the outcome, this function redirects the browser to the landing page.

READ - Get a specific user

```
@app.route('/read_user/<int:user_id>') ①
def read_user(user_id):
    user = User.query.get_or_404(user_id) ②
    return f"User: {user.username}, Email: {user.email}" ③
```

① This endpoint requires a integer user id to be passed.

2. If the user is not found, this function returns a HTTP error code 404 "Not Found".
3. If the user is found, this function returns the string similar to the hello-world example, not a full HTML template.

UPDATE - Modify a user

```
@app.route('/update_user/<int:user_id>/<string:username>/<string:email>')  1
def update_user(user_id, username, email):
    user = User.query.get_or_404(user_id)
    user.username = username  2
    user.email = email
    try:
        db.session.commit()
    except Exception as e:
        db.session.rollback()
    return redirect(url_for('index'))
```

1. This endpoint requires the user ID of the entry that the visitor wants to change as well as the new username and email.
2. Here the values are updated in the User object without committing the changes yet.

DELETE - Remove a user

```
@app.route('/delete_user/<int:user_id>')
def delete_user(user_id):
    user = User.query.get_or_404(user_id)
    try:
        db.session.delete(user)
        db.session.commit()
    except Exception as e:
        db.session.rollback()
    return redirect(url_for('index'))
```

Putting it all together

With the code snippets above you can now create a server that lets clients manipulate the database. The complete `app.py` will then look as follows.

> **Note the required `redirect`, `url_for`, and `flash` imports from the flask package in line 1**
>
> - `redirect` lets you send the client to another route (e.g., to the user profile page after adding a new user).
> - `url_for` gives you the url for a route (e.g., index); you have used these in the previous section already in the HTML templates.
> - `flash` is used to send pop-up notifications from the server to the client.

app.py

```python
from flask import Flask, render_template, redirect, url_for, flash
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all()

@app.route('/')
def index():
    users = User.query.all()
    return render_template('home.html', users=users)

@app.route('/add_user/<string:username>/<string:email>')
def add_user(username, email):
    new_user = User(username=username, email=email)
    try:
        db.session.add(new_user)
        db.session.commit()
        flash(f'User {username} added successfully!', 'success')
    except Exception as e:
        db.session.rollback()
        flash(f'Error: {str(e)}', 'error')
    return redirect(url_for('index'))

@app.route('/read_user/<int:user_id>')
def read_user(user_id):
    user = User.query.get_or_404(user_id)
    return f"User: {user.username}, Email: {user.email}"

@app.route('/update_user/<int:user_id>/<string:username>/<string:email>')
def update_user(user_id, username, email):
    user = User.query.get_or_404(user_id)
    user.username = username
    user.email = email
    try:
        db.session.commit()
        flash(f'User {username} updated successfully!', 'success')
    except Exception as e:
        db.session.rollback()
        flash(f'Error: {str(e)}', 'error')
    return redirect(url_for('index'))

@app.route('/delete_user/<int:user_id>')
def delete_user(user_id):
    user = User.query.get_or_404(user_id)
    try:
        db.session.delete(user)
        db.session.commit()
        flash(f'User deleted successfully!', 'success')
    except Exception as e:
        db.session.rollback()
```

```
        flash(f'Error: {str(e)}', 'error')
    return redirect(url_for('index'))
```

You can add a user by typing directing your browser to the URL http://127.0.0.1:5000/add_user/jane/jane@asu.edu. It will run the `add_user` function on the server and redirect you to the homepage. When the homepage is loaded, it will get all the users and you should see the new entry in the list.

You should put nicer error messages into production

The code above injects the raw python error message into the notification that is sent to the user. You should write something that is more helpful to the user.

Next steps to try it out!

- Try out the update and delete endpoints as well.
- Check your manipulations in the MySQL Workbench.
- Try to cause an error message to show up.
- Try adding Bootstrap and custom styling to the flash notifications.